The file, “health_check.sh,” is a Bash (or POSIX) shell script that checks CPU, memory, and disk usage, logs the results, and sends email alerts if defined thresholds are exceeded. This guide explains how to prepare a Gmail account and Postfix setup to enable email delivery. To run the file “health_check.sh”, navigate to its directory location or folder, open a terminal and type “**./health_check.sh**”. An output file or log file, “health.log” will be generated in the same folder. Read the log file with any simple text editor.

Certain prerequisites must be in place for the file, “health_check.sh”, to function.

- You must have **2-Step Verification** enabled on the recipient Google* account.
- You must be logged into your Gmail account.

* This example was done using a Google email account. Equivalent steps must be taken for other email servers.

For the script to send emails to the recipient email address, an app password must be created. This app password must be made known to the host computer running the script that sends the alert emails to the recipient. The app password is an alternative password to the recipient email account that gives the it the ability to send emails to itself containing the script’s alert messages. As such, the app password must not be shared with others as it gives full access to sending email ***from the recipient*** ***account***. The app password can be ***revoked or deleted at any time*** from the App Passwords page (figure 1).

Steps to Generate an App Password in Google:

Step 1: Go to Google App Passwords page

- Visit: https://myaccount.google.com/apppasswords

Step 2: Verify your identity

- Google may ask you to log in again or enter a verification code.

Step 3: Choose the app and device (or name it)

- In the drop-down or text box, type a unique, easy-to-remember name like:

“swift”, “CopyEdit” or “postfix-server”.
This helps you remember what the password is for.

Step 4: Click “Create”

- Google will generate a **16-character password** (example: abcd efgh ijkl mnop).

Step 5: Copy that app password

- Save it temporarily — you'll use it in your Postfix configuration.
- **Important**: You will not see it again after this screen.
- When completed the app password will be seen on the App passwords page (figure 1):

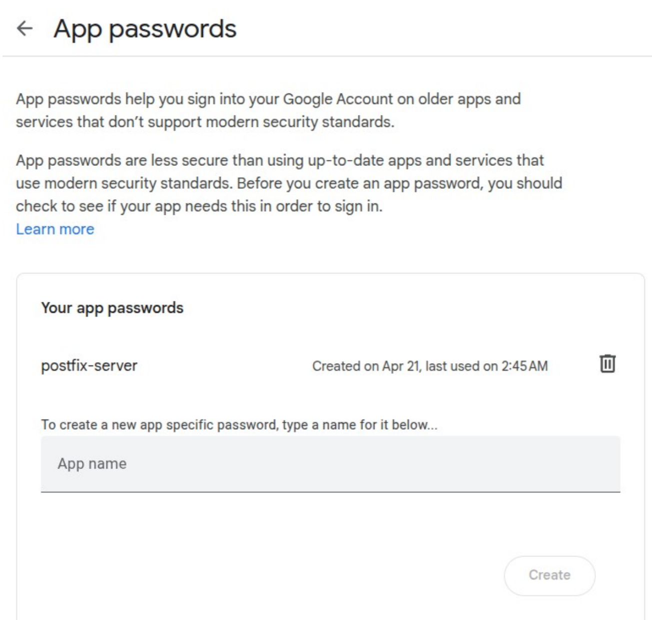

Figure 1. Screen capture of Google App passwords page showing app passwords.

In the final step, Postfix must be given access to the app password in order to have the recipient email account relay script alert emails to itself (the recipient (Google) email address). This allows Postfix to authenticate with the Google mail server. This is accomplished by configuring Postfix's Simple Mail Transfer Protocol (SMTP) authentication settings. This will involve modifying the file,
"/etc/postfix/sasl_passwd"
to include the line,
"[smtp.gmail.com]:587   the_recipient_email@gmail.com:the_saved_16_char_app_password", where
"[smtp.gmail.com]:587 " → tells Postfix to use port 587 to contact the Gmail SMTP server.
"the recipient_email@gmail.com" → identifies the recipient email address
"the_saved_16_char_app_password" → identifies the app password
The file, "/etc/postfix/sasl_passwd", is where Postfix looks for SMTP login credentials. Note that there is no space between the recipient email address and the app password above.

Step 6: Use the app password in Postfix

- Access the file, "/etc/postfix/sasl_passwd", from the terminal using the superuser (sudo) command, the nano simple text editor (nano), and the file directory path, "/etc/postfix/sasl_passwd", in a single line.
- When prompted because of sudo, enter the system or user password.
- The file, "/etc/postfix/sasl_passwd", will be opened by nano as shown below. (NOTE: app password has been blacked out, [redacted] , due to previously mentioned security risks)

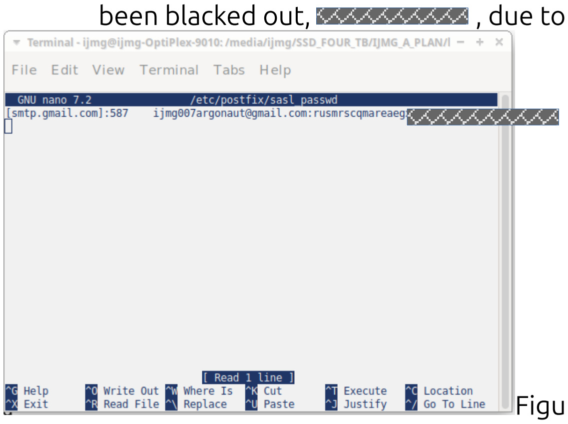

Figure 2. Nano used to open file, "/etc/postfix/sasl_passwd".

- The line, "[smtp.gmail.com]:587   the_recipient_email@gmail.com:the_saved_16_char_app_password" was cut and pasted directly into the file, "/etc/postfix/sasl_passwd", as shown above in figure 2. When pasting the line, don't include quotes, brackets, or extra spaces — it must be exactly one line, tab or space-separated.
- To save the changes in nano, type "Ctrl + O" to save the file changes, followed by "Enter" to confirm the file name, followed by "Ctrl + X" to exit nano, then return to the terminal.
- Next, type the following commands into the terminal one at a time:

Command	Function
sudo postmap /etc/postfix/sasl_passwd	Converts the **plain-text password file** into a special hashed version Post-fix can read called, "/etc/postfix/sasl_passwd.db"
sudo chmod 600 /etc/postfix/sasl_passwd	Changes the file permissions so that **only the root user can read/write** the file (important due to app password contents). "600" means Owner *only* (root): **read + write.**
sudo systemctl restart postfix	Tells the system to **restart the Postfix mail service**, reloading its configuration and updated credential file.

- Post-fix should now be ready to access the recipient email account for relay of alert messages.

Step 7: Verification of function

- The recipient email account should show results similar to below in figure 3.

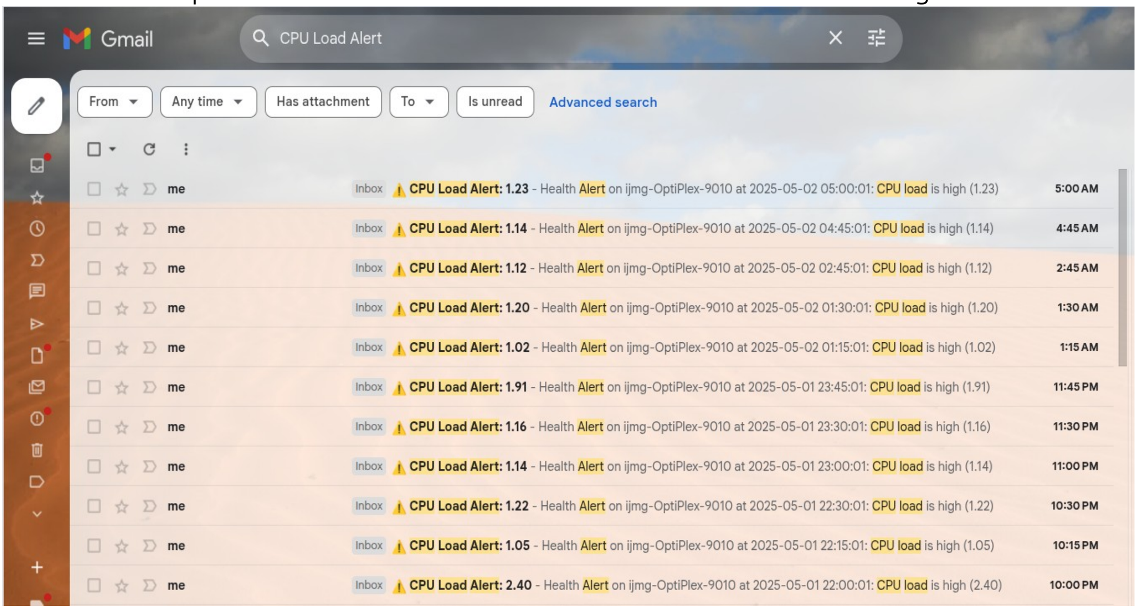

Figure 3. Relayed Gmail alert emails.

Below are some trouble shooting steps for failed email relays.

Step	Command	What to Look For	Why	Fix
Check mail logs	sudo journalctl -u postfix -n 50	Timeouts, auth errors, missing relay attempts	To see if emails are being attempted or failing silently	Identify if issue is authentication, TLS, or connectivity
Check if mail command exists	which mail	Path like /usr/bin/mail	Confirms that a working mail command is installed	Install mailutils if missing
Test mail command	echo "test" | mail -s "Test" user@example.com	Whether email is received	To verify email path works end-to-end	Check config/logs if not received
Install correct mail agent	sudo apt purge bsd-mailx && sudo apt install mailutils	Successful install	bsd-mailx lacks proper SMTP support	Use mailutils which integrates with Postfix
Review mail.log after test	sudo tail -n 50 /var/log/mail.log	Connection timeouts, port 25 usage	To see real errors like blocked port 25	Switch to port 587 with STARTTLS
Fix TLS config	sudo nano /etc/postfix/main.cf	relayhost, smtp_use_tls, smtp_sasl_* options	Postfix may default to port 25 without STARTTLS	Add correct Gmail SMTP lines + restart Postfix

Table 1. Trouble shooting for failed email relays.

This completes the setup for automated server health checks with email alerting using Bash, Postfix, and Gmail.